
OPENCV DESDE PYTHON:

CALIBRACIÓN DE HOMOGRAFÍAS CON OPENCV

INTRODUCCIÓN

OpenCV dispone de varios métodos robustos de calibración de homografías. En el primer proyecto de esta asignatura, se deberá calibrar la homografía de una cámara respecto a un plano de trabajo que permitirá posicionar los centros de los objetos reales sobre el plano de trabajo a partir de sus centroides en las imágenes.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es aprender a posicionar objetos de interés sobre un plano, partiendo de la detección de sus centroides en una imagen capturada por una cámara. Para ello, se podrán utilizar diferentes métodos robustos de calibración de homografías utilizando OpenCV desde Python. Además, en esta práctica se muestra un ejemplo de cómo gestionar eventos del ratón para seleccionar un pixel de una imagen utilizando OpenCV desde Python.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/60-Jupyter lab tutorial – Homography
- PoliformaT -> Recursos/2-Software-&-cuadernos/61-Jupyter lab tutorial - Homography calibration

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

```
(base) C:\windows\system32> conda activate open3d_env
```

```
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working\_folder\_path”
```

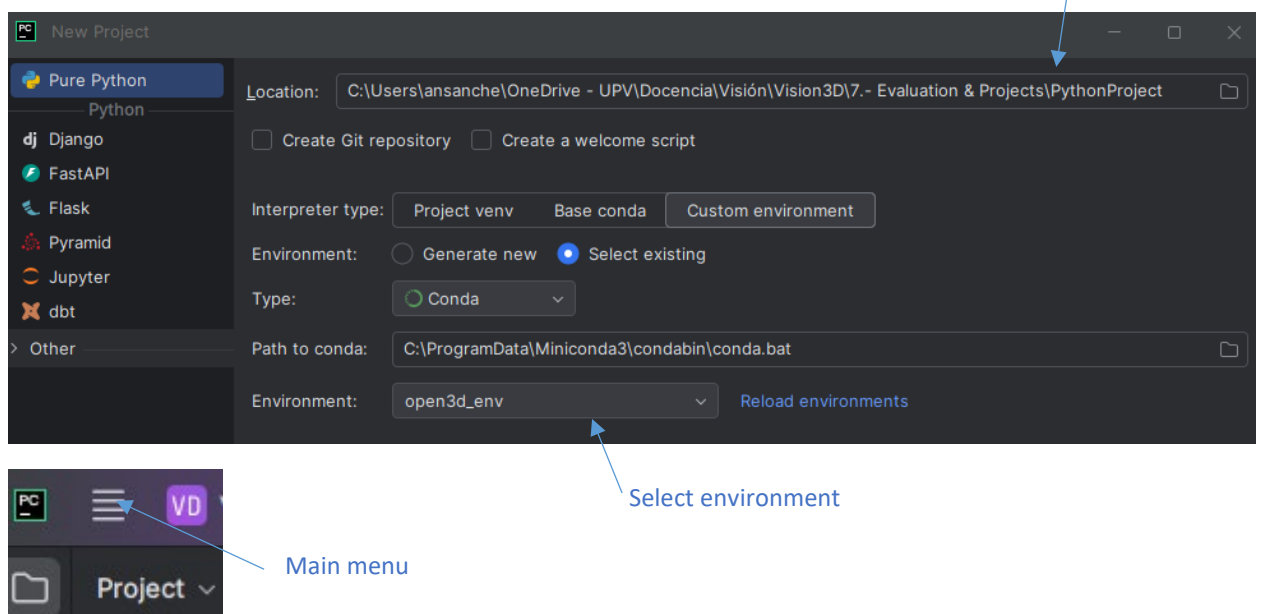
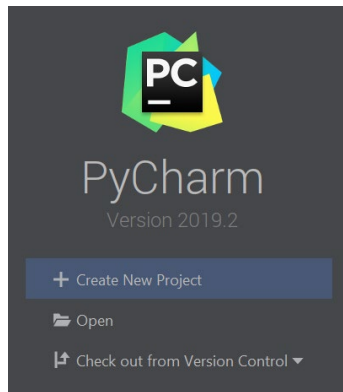
Después de arrancar Jupyter lab, escribe en una célula de código el comando: `"import cv2"`

Mediante este módulo, Python encontrará todos los métodos de Python que sirven de interfaz entre la librería OpenCV de C++ y Python.

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

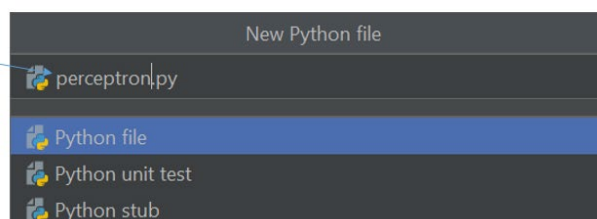
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



EJERCICIO 1: ENTENDER CÓDIGO DE COMPARACIÓN DE MÉTODOS:

```
[ ]: # Defining homography
H = np.array([[0.55, -0.2, 100], [0.1, 0.8, 50], [1e-5, 1e-4, 1]])

# generate N random points, transform them using a reference H
N = 20
p1 = np.random.randint([480, 640], size=(N, 2))
p1 = p1.astype(np.float32).reshape(-1,1,2)
p2 = cv2.perspectiveTransform(p1, H)

# introduce small noise in points
p1 = p1 + np.random.randn(p1.shape[0], p1.shape[1], p1.shape[2])
p2 = p2 + np.random.randn(p2.shape[0], p2.shape[1], p2.shape[2])

# introduce an extreme outlier (to test robust estimators)
p2[0,0] = 1000

# estimate homography by using four different methods
estMethods = [ 0, cv2.RANSAC, cv2.LMEDS, cv2.RHO ]
errs=[]
for method in estMethods:
    HH, mask = cv2.findHomography(p1, p2, method, 3.0, None, 2000, 0.995)
    mask = mask == 1
    p1_act = p1[mask].reshape(-1,1,2)
    p2_act = p2[mask].reshape(-1,1,2)

    p2_est = cv2.perspectiveTransform(p1_act, HH)
    if p2_act.shape[0] > 0:
        err = cv2.norm(p2_act, p2_est, cv2.NORM_L2)/p2_act.shape[0]
        errs.append(err)

print('Mean re-projection errors:', errs)
```

EJERCICIO 2: APLICAR LA HOMOGRAFÍA

A partir de las imágenes fish y fish_perspective obtener la homografía que las relaciona. Para ello obtener X puntos en la imagen 1 y sus correspondientes puntos en la imagen 2. Una vez obtenida la matriz homografía comprobar su validez restaurando la imagen con perspectiva con la siguiente función:

```
imagenDestino = cv2.warpPerspective(imagen, matrizHH, None)
```

Nota: Se puede automatizar la obtención de puntos con el ratón:

```
def draw_circle(event,x,y,flags,param):
    global mouseX,mouseY
    if event == cv2.EVENT_LBUTTONDOWN:
        cv2.circle(img, (x,y), 10, (0,0,255), 2)
        mouseX,mouseY = x,y

def get_points(filename):
    global img
    img = cv2.imread(filename)

    pts = []
    cv2.namedWindow('img')
    cv2.setMouseCallback('img', draw_circle)
    while(1):
        cv2.imshow('img',img)
        k = cv2.waitKey(20) & 0xFF
        if k == 27:
            break
        elif k == ord('a'):
            pts.append([mouseX,mouseY])
            print(mouseX,mouseY)

    cv2.destroyAllWindows() # Closes the window
    return pts
```

EJERCICIO 3: DETECCIÓN DE PUNTOS AUTOMÁTICA

Calcular la homografía entre la imagen Lena y Lena_perspective obteniendo los puntos de forma automática con las siguientes funciones:

```
%Funciones para calcular descriptores de puntos de interés de ORB
MAX_FEATURES = 100
obj = cv2.ORB_create(MAX_FEATURES)
[kpts1, feat1] = obj.detectAndCompute(im1, None)

%Funciones para calcular que puntos corresponden entre sí matcher
= cv2.DescriptorMatcher_create(cv2.
    ,cv2.DEScriptorMatcher_BRUTEFORCE_HAMMING)
matches = matcher.match(feat1, feat2, None)

%Función para dibujar los puntos asociados entre imágenes
out = cv2.drawMatches(im1, kpts1, im2, kpts2, matches, None)
```

Nota: "kpts1(m(i).queryIdx).pt" es el punto (X,Y) de la imagen 1 que se corresponde con el punto de la imagen 2: "kpts2(m(i).trainIdx).pt";

4.-TAREA

A partir de los materiales y ejemplos anteriores deberéis integrar en vuestro proyecto:

- un método que calibre la homografía de vuestra cámara con respecto a vuestro plano de trabajo (intentar estimar la homografía de la manera más precisa posible),
- además de un método que posicione (o reconstruya) sobre el plano de trabajo cualquier pixel de la imagen capturada.